Course Code: 2165119 **Course Name:** ASD&D Lab

Class: TE **Batch:** T23 **Semester:** V

Student Name: Tejas Mandhyani **Roll No: 2511135**

—----------

EXPERIMENT NO. 04

Aim: - To set up a Continuous Integration Pipeline using GitHub Actions.

Theory:

Continuous Integration (CI) is a software development practice where developers regularly merge their code changes into a central repository, after which automated builds and tests are run. The key goals of CI are to find and address bugs quicker, improve software quality, and reduce the time it takes to validate and release new software updates.

GitHub Actions is a cloud-based CI/CD platform that allows developers to automate their build, test, and deployment pipelines directly within GitHub.

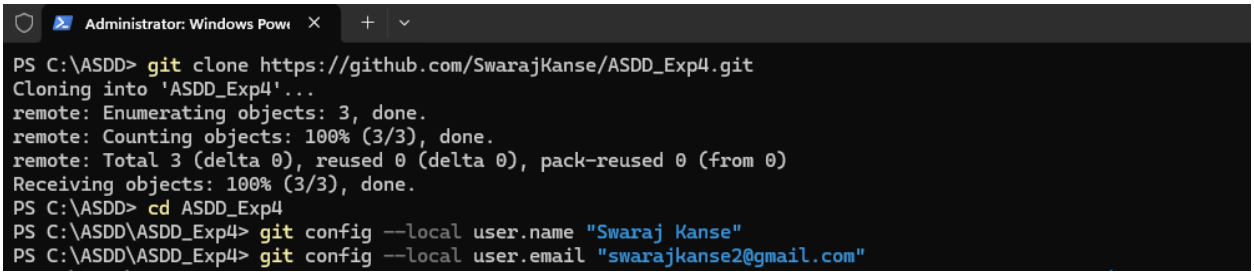
Core Concepts of GitHub Actions:

1. **Workflows:** A configurable automated process that will run one or more jobs. Workflows are defined by a YAML file checked into your repository under the .github/workflows directory.
2. **Events:** A specific activity in a repository that triggers a workflow run (e.g., a push to the main branch or a pull request).
3. **Jobs:** A set of steps in a workflow that execute on the same runner. Each step is either a shell script that will be executed, or an action that will be run.
4. **Runners:** A server that runs your workflows when they're triggered (e.g., an Ubuntu Linux machine hosted by GitHub).

IMPLEMENTATION

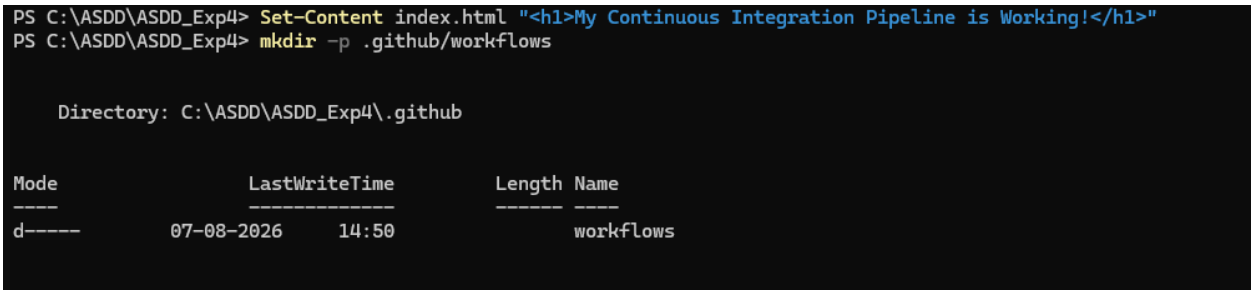
Step 1: Cloning the Repository and Setting up the Workspace

First, a remote GitHub repository named ASDD_Exp4 was created. Then, it was cloned to the local lab environment and user credentials were configured locally.



Step 2: Creating Application Code and Workflow Directory

A sample HTML file was created to act as the application code, and the required .github/workflows directory structure was generated for the pipeline configuration.



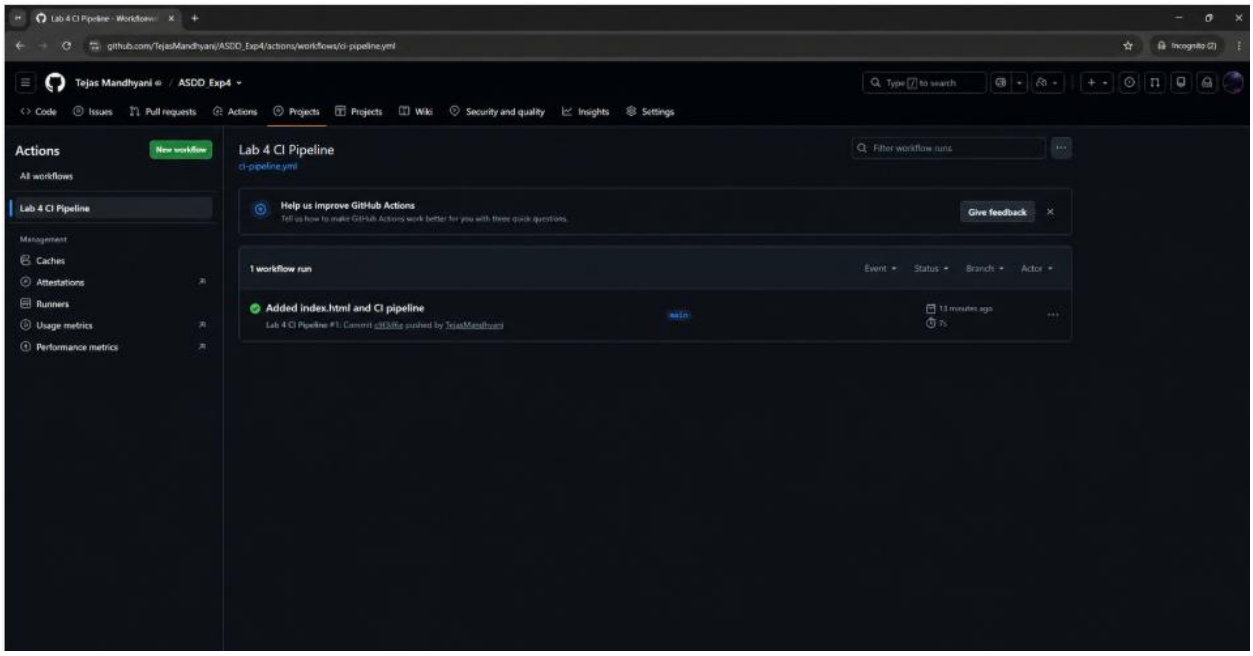
Step 3: Defining and Pushing the CI Pipeline

The pipeline YAML file (ci-pipeline.yml) was created using Notepad, defining a workflow that runs on ubuntu-latest and executes Build and Test phases whenever code is pushed to the main branch. The changes were then staged, committed, and pushed to GitHub.

```
PS C:\ASDD\ASDD_Exp4> notepad .github/workflows/ci-pipeline.yml
PS C:\ASDD\ASDD_Exp4> git add .
PS C:\ASDD\ASDD_Exp4> git commit -m "Added index.html and CI pipeline"
[main cd63b96] Added index.html and CI pipeline
 2 files changed, 26 insertions(+)
 create mode 100644 .github/workflows/ci-pipeline.yml
 create mode 100644 index.html
PS C:\ASDD\ASDD_Exp4> git push origin main
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 20 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (6/6), 757 bytes | 757.00 KiB/s, done.
Total 6 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/SwarajKanse/ASDD_Exp4.git
   9444327..cd63b96  main -> main
PS C:\ASDD\ASDD_Exp4>
```

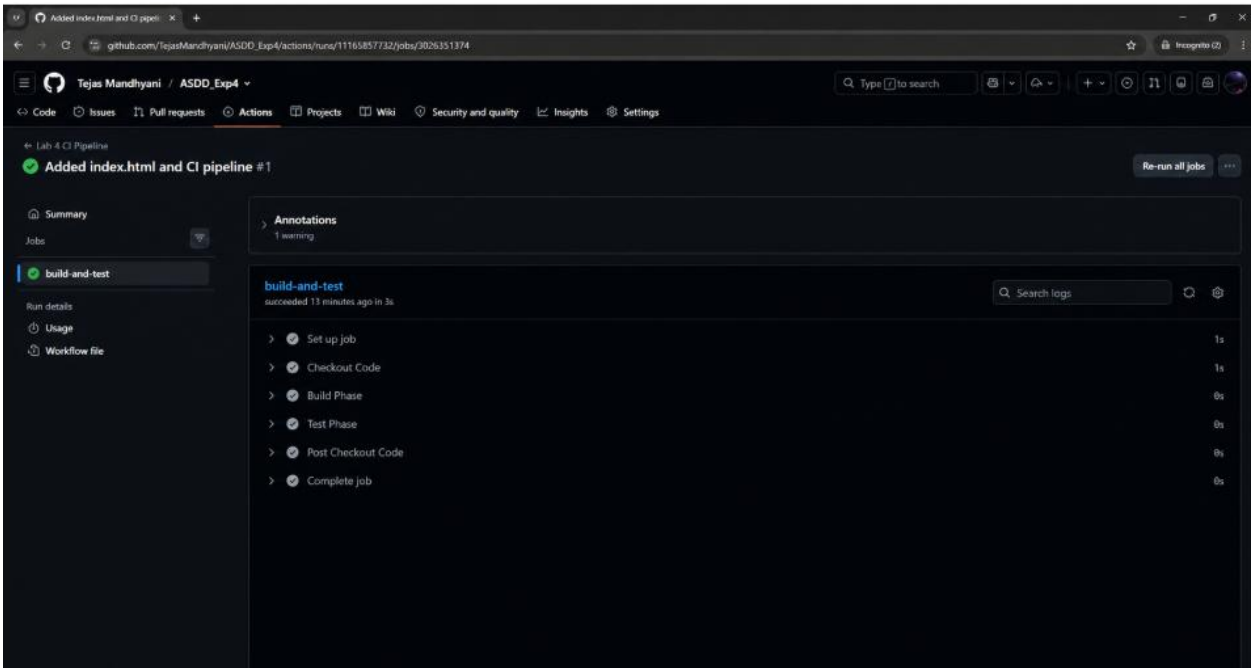
Step 4: Verifying Pipeline Execution on GitHub Actions

Upon pushing the code, GitHub Actions automatically intercepted the push event and triggered the workflow. Navigating to the "Actions" tab on the GitHub repository showed that the workflow "Added index.html and CI pipeline" executed successfully with a green checkmark.



Step 5: Reviewing the Build and Test Logs

Clicking into the build-and-test job detailed the live logs of the runner. The logs confirmed that the runner successfully checked out the code, executed the custom Build Phase, and passed the automated Test Phase as configured in the YAML file.



Conclusion: Thus, we have successfully studied the concepts of Continuous Integration (CI) and configured an automated CI pipeline using GitHub Actions to automatically build and test application code upon repository push events.